SCH- MGMT 661:
Applications of Artificial Intelligence Models

Transfer Learning

Isenberg School of Management

Instructor: Indika Dissanayake

Agenda

- What is Transfer Learning?
- Why It Matters in Business Applications
- Transfer Learning Approaches
- Workflow Overview
- Image Resizing and Data Augmentation
- Learning Rate Scheduling and Early Stopping
- Case: Transfer Learning with ImageNet (MobileNet2)

Transfer Learning

- **Transfer Learning**: Reusing a model trained on a large dataset and adapting it to a new, smaller task.
- **Why use it?**
  - Leverage pre-learned features from big datasets
  - Faster training – only a few layers need learning
  - Lower risk of overfitting – the base model is already well-regularized
  - *e.g., Using a model pre-trained on millions of everyday images (ImageNet) as a starting point to build a new model for classif.ying new categories like 'types of vehicles' or 'product defects.*

Real-World Applications

- Medical Imaging (using pretrained ResNet or EfficientNet for X-rays, CT scans)
- E-commerce (classifying product photos)
- Quality Inspection (detecting defects in manufacturing)
- Satellite Imagery (land use and vegetation analysis)
- Social Media (content tagging, moderation)
- Autonomous Vehicles (Object and lane detection using pretrained models)

Transfer Learning Approaches

Feature Extraction (reuse features)

- Use a pretrained model (e.g., VGG16, ResNet50) as a fixed feature extractor
- Freeze the base (convolutional) layers to keep pretrained weights fixed
- Replace the classifier (Dense layers or “head”) with a new one for your dataset
- Best for smaller datasets and faster results when your new task is similar to the original

Fine-Tuning (reuse + adapt features)

- Unfreeze a few top layers of the pretrained model (base model)
- Retrain those layers and the head with your data
- Allows adaptation of learned features to your specific dataset
- Slower but may improve performance when datasets differ significantly

Transfer Learning Workflow

- **Step-by-step Workflow:**
  a. **Load pretrained model** (e.g., MobileNet, ResNet – trained on ImageNet)
  b. **Freeze Base Layers** – preserved learned filters for low-level features
  c. **Add New Classification Head** – dense layer customized for your task
  d. **Train Only New Layers** – feature extraction phase
  e. **(*Optional*) Unfreeze and Fine-Tune Top Layers** – with a low learning rate, few epochs
  f. **Evaluate Performance** – compare results before and after fine-tuning

Image Resizing in Transfer Learning

- Why Resize?
  - Pretrained models expect fixed input sizes (e.g., 96×96, 128×128, or 224×224)
  - Datasets like CIFAR-10 (32×32) are much smaller
  - Resizing ensures compatibility with pretrained architectures.
    - e.g., MobileNetV2 typically uses 96×96 or 128×128 pixels.

Data Augmentation

- To prevent overfitting and improve generalization:
  - Random horizontal flips
  - Small rotations and shifts
  - Brightness or zoom variations

*This layer can be added **before the base model** inside the pipeline.*

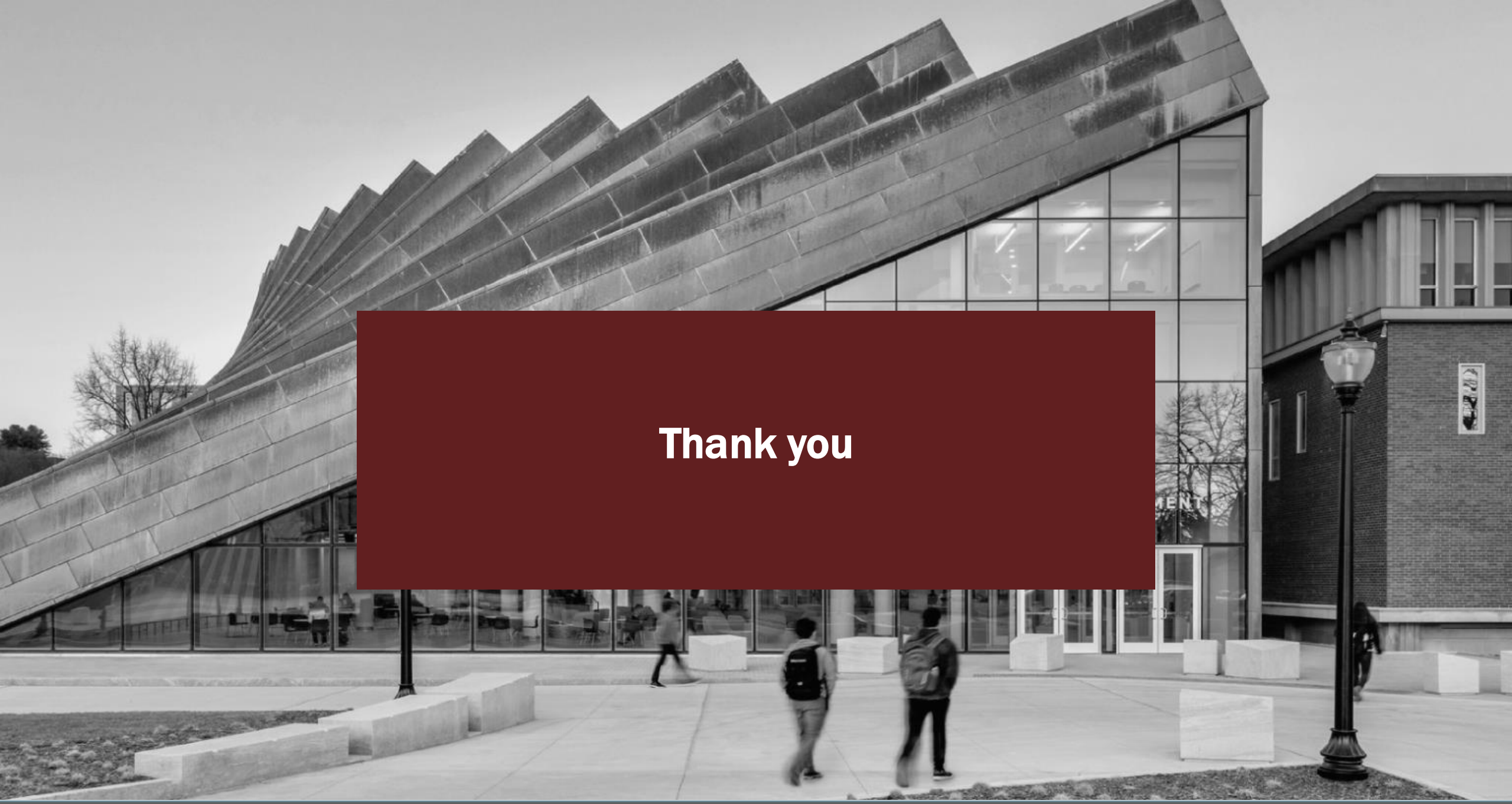
```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.05), # 5% rotation
    layers.RandomZoom(0.1),      # 10% zoom
    layers.RandomContrast(0.2),  # 20% contrast variation

])
```

Learning Rate Scheduling & Early Stopping

- Why use callbacks?
  - To make training smarter and more efficient.
  - To automatically adjust learning and stop training early to prevent overfitting or wasted computation.
  - To improve convergence stability
- What happens
  - ReduceLROnPlateau: Lowers learning rate when validation loss stops improving.
  - EarlyStopping: Stops training if no improvement is seen for a few epochs, and restores the best weights.
- .

```
# Add callbacks for better training
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        patience=3, restore_best_weights=True
        ),
    tf.keras.callbacks.ReduceLROnPlateau(
        factor=0.5, patience=2
        )
]
```



Thank you

Reference

- https://www.tensorflow.org/tutorials/images/transfer_learning
- https://keras.io/api/applications/